

http://algs4.cs.princeton.edu

2.4 PRIORITY QUEUES

- *API and elementary implementations*
- *binary heaps*
- *heapsort*
- *event-driven simulation*



2.4 Priority Queues

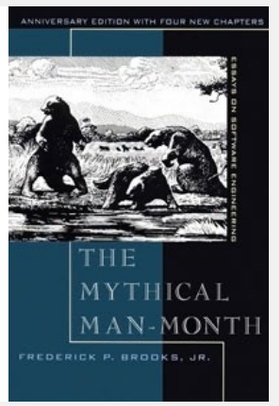
- *API and elementary implementations*
- *binary heaps*
- *heapsort*
- *event-driven simulation*

Collections

Një collection është një tip i të dhënave që ruan një grup të elementeve (items).

data type	key operations	data structure
stack	PUSH, POP	*linked list, resizing array*
queue	ENQUEUE, DEQUEUE	*linked list, resizing array*
priority queue	INSERT, DELETE-MAX	*binary heap*
symbol table	PUT, GET, DELETE	*BST, hash table*
set	ADD, CONTAINS, DELETE	*BST, hash table*

“ Show me your code and conceal your data structures, and I shall continue to be mystified. Show me your data structures, and I won't usually need your code; it'll be obvious.” — *Fred Brooks*



Priority queue

Collections. Insert dhe delete elemente.

Stack. Remove elementin e fundit të shtuar.

Queue. Remove elementin e pare të shtuar.

Randomized queue. Remove një random element.

Priority queue. Remove elementin më të madh (ose më të vogël).

operation	*argument*	*return value*
insert	P	
insert	Q	
insert	E	
remove max		Q
insert	X	
insert	A	
insert	M	
remove max		X
insert	P	
insert	L	
insert	E	
remove max		P

Priority queue API

Requirement. Elementet Generike janë `Comparable`.

Çelësi duhet të jetë Comparable (bounded type parameter)

`public class MaxPQ<Key extends Comparable<Key>>`

	`MaxPQ()`	*create an empty priority queue*
	`MaxPQ(Key[] a)`	*create a priority queue with given keys*
`void`	`insert(Key v)`	*insert a key into the priority queue*
`Key`	`delMax()`	*return and remove the largest key*
`boolean`	`isEmpty()`	*is the priority queue empty?*
`Key`	`max()`	*return the largest key*
`int`	`size()`	*number of entries in the priority queue*

Priority queue - aplikimet

- Event-driven simulation. [customers in a line, colliding particles]
- Numerical computation. [reducing roundoff error]
- Data compression. [Huffman codes]
- Graph searching. [Dijkstra's algorithm, Prim's algorithm]
- Number theory. [sum of powers]
- Artificial intelligence. [A* search]
- Statistics. [online median in data stream]
- Operating systems. [load balancing, interrupt handling]
- Computer networks. [web cache]
- Discrete optimization. [bin packing, scheduling]
- Spam filtering. [Bayesian spam filter]

Generalizon: stack, queue, randomized queue.

Priority queue klienti: shembull

Sfida. Gjindet elementi më i madh M në një stream prej N elementeve.

implementation	time	space
sort	$N \log N$	N
elementary PQ	$M\,N$	M
binary heap	$N \log M$	M
best in theory	N	M

order of growth për të gjetur M-në më të madhe në një stream prej N elementeve

Priority queue: Implementimi i array-ave të sortuara dhe pasortuara

operation	*argument*	*return value*	*size*	*contents (unordered)*							*contents (ordered)*						
insert	P		1	P							P						
insert	Q		2	P	Q						P	Q					
insert	E		3	P	Q	E					E	P	Q				
remove max		Q	2	P	E						E	P					
insert	X		3	P	E	X					E	P	X				
insert	A		4	P	E	X	A				A	E	P	X			
insert	M		5	P	E	X	A	M			A	E	M	P	X		
remove max		X	4	P	E	M	A				A	E	M	P			
insert	P		5	P	E	M	A	P			A	E	M	P	P		
insert	L		6	P	E	M	A	P	L		A	E	L	M	P	P	
insert	E		7	P	E	M	A	P	L	E	A	E	E	L	M	P	P
remove max		P	6	E	M	A	P	L	E		A	E	E	L	M	P	

Sekuenca e operacioneve në një priority queue

Priority queue: Implementimi i array të pasortuar

```
public class UnorderedArrayMaxPQ<Key extends Comparable<Key>>
{
   private Key[] pq;    // pq[i] = ith element on pq
   private int N;       // number of elements on pq

   public UnorderedArrayMaxPQ(int capacity)
   {  pq = (Key[]) new Comparable[capacity];  }

   public boolean isEmpty()
   {  return N == 0; }

   public void insert(Key x)
   {  pq[N++] = x;  }

   public Key delMax()
   {
      int max = 0;
      for (int i = 1; i < N; i++)
         if (less(max, i)) max = i;
      exch(max, N-1);
      return pq[--N];
   }
}
```

no generic array creation

less() and exch() similar to sorting methods (but don't pass pq[])

should null out entry to prevent loitering

Priority queue – implementimi bazik

Sfida. Të implement të gjitha operacionet me efikasitet.

implementation	insert	del max	max
unordered array	1	N	N
ordered array	N	1	1
goal	$\log N$	$\log N$	$\log N$

order of growth për running time për priority queue me N elemente



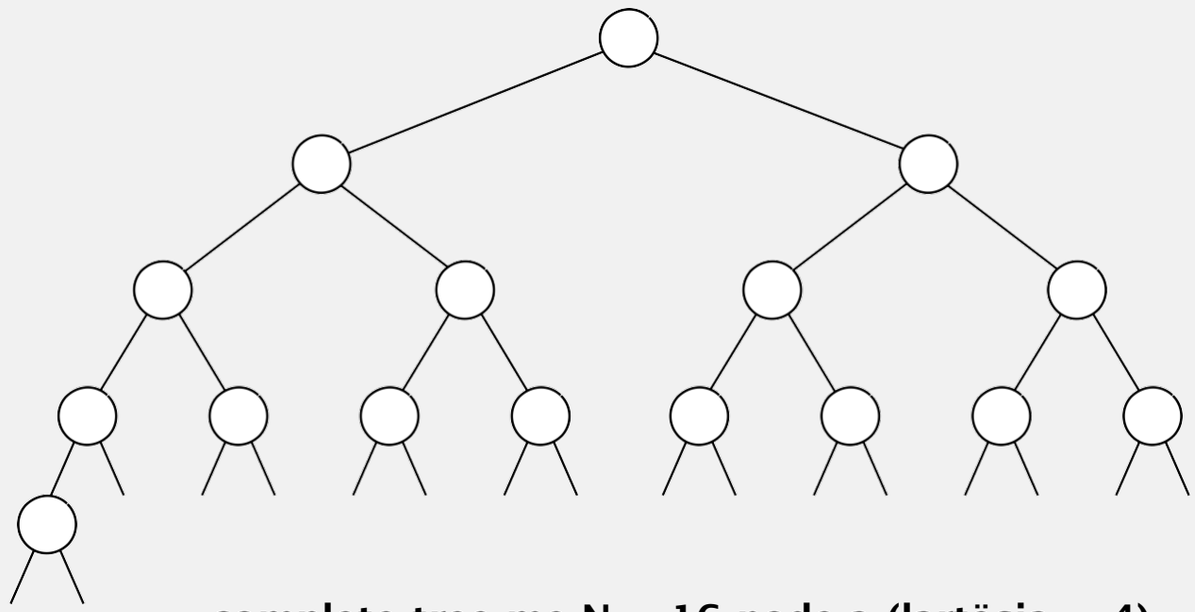
2.4 Priority Queues

- *API and elementary implementations*
- *binary heaps*
- *heapsort*
- *event-driven simulation*

Complete binary tree

Binary tree. Empty ose node me lidhje majtas dhe djathtas me binary trees.

Complete tree. Plotësisht i balancuar (plotësuar), përveç në nivelin e fundit.



complete tree me N = 16 node-a (lartësia = 4)

Vetitë. Lartësia e complete tree me N node-a është $\lg N$.

Vërtetim. Lartësia rritet vetëm kur N është në fuqi të2.

Complete binary tree në natyrë



Binary heap - paraqitja

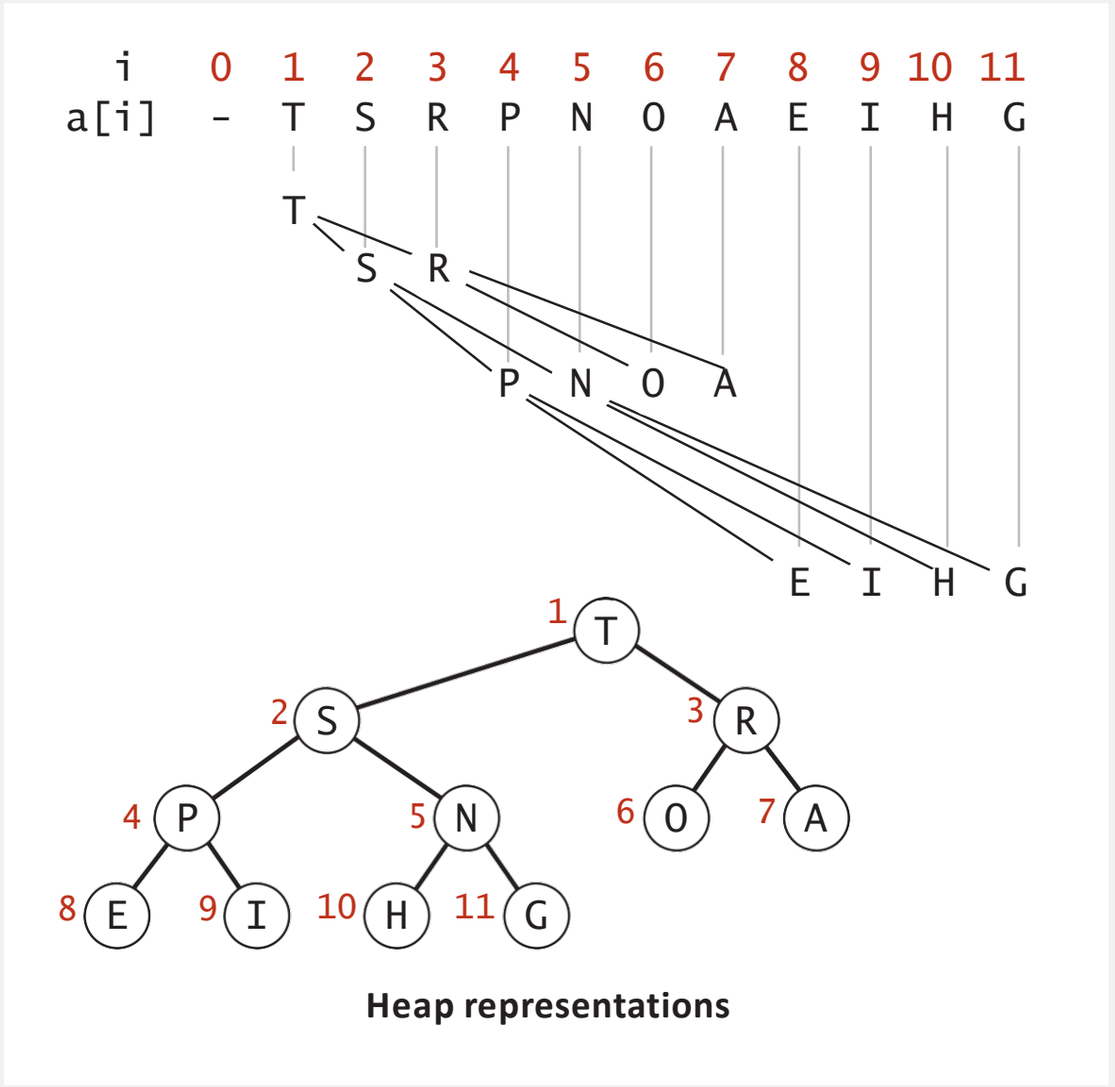
Binary heap. Paraqitja e array e një heap-ordered complete binary tree.

Heap-ordered binary tree.

- Çelësat në nyje.
- Çelësi I prindërve nuk mund të jetë më i vogël sesa ai i fëmijëve.

Paraqitja me array.

- Treguesi starton tek 1.
- Mirret nyja dhe renditet në nivelin përkatës.
- Nuk ka linka medoemos



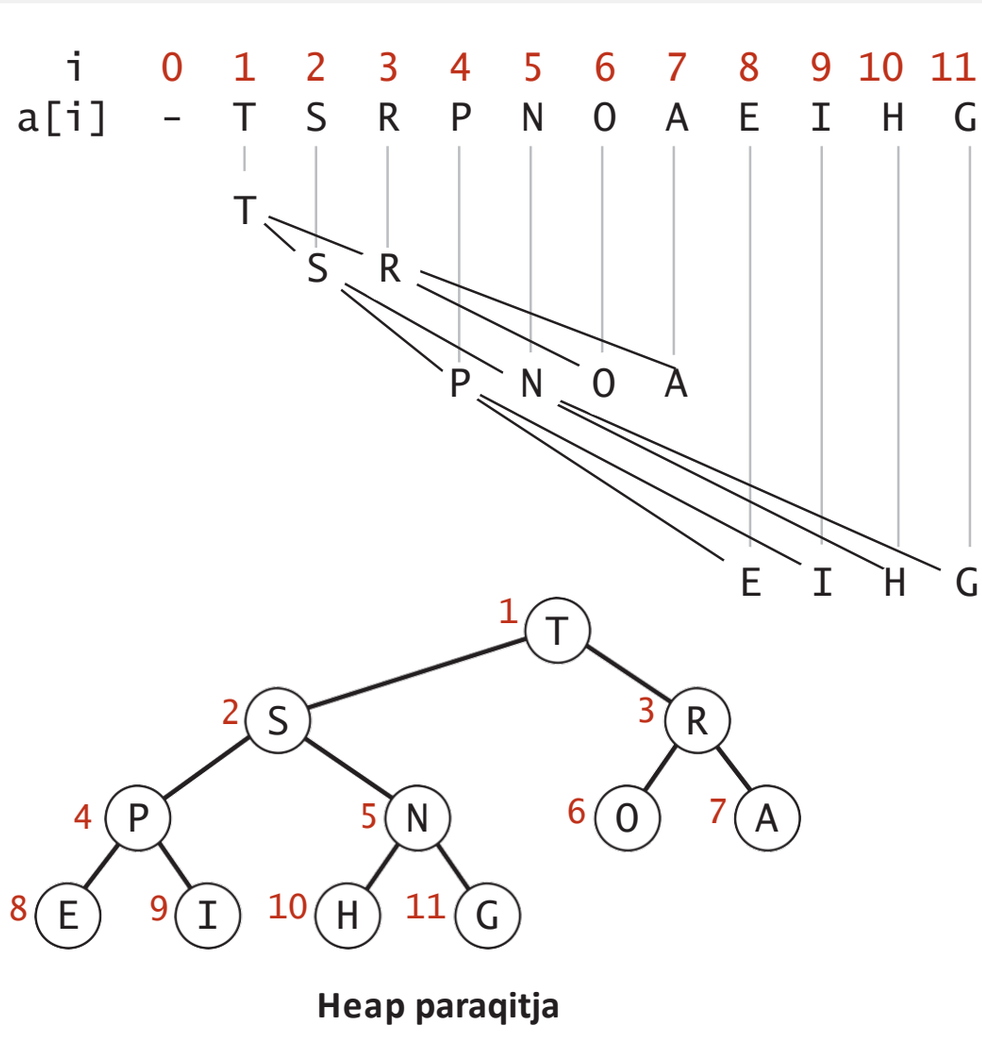
Heap representations

Binary heap - vetitë

Teoremë. Çelësi më i madh është `a[1]`, që është root i një binary tree.

Teoremë. Mund të përdoren treguesit e array për të lëvizur nëpër tree.

- Prindi i nyjës `k` është `k/2`.
- Fëmijët e nyjës `k` janë `2k` dhe `2k+1`.



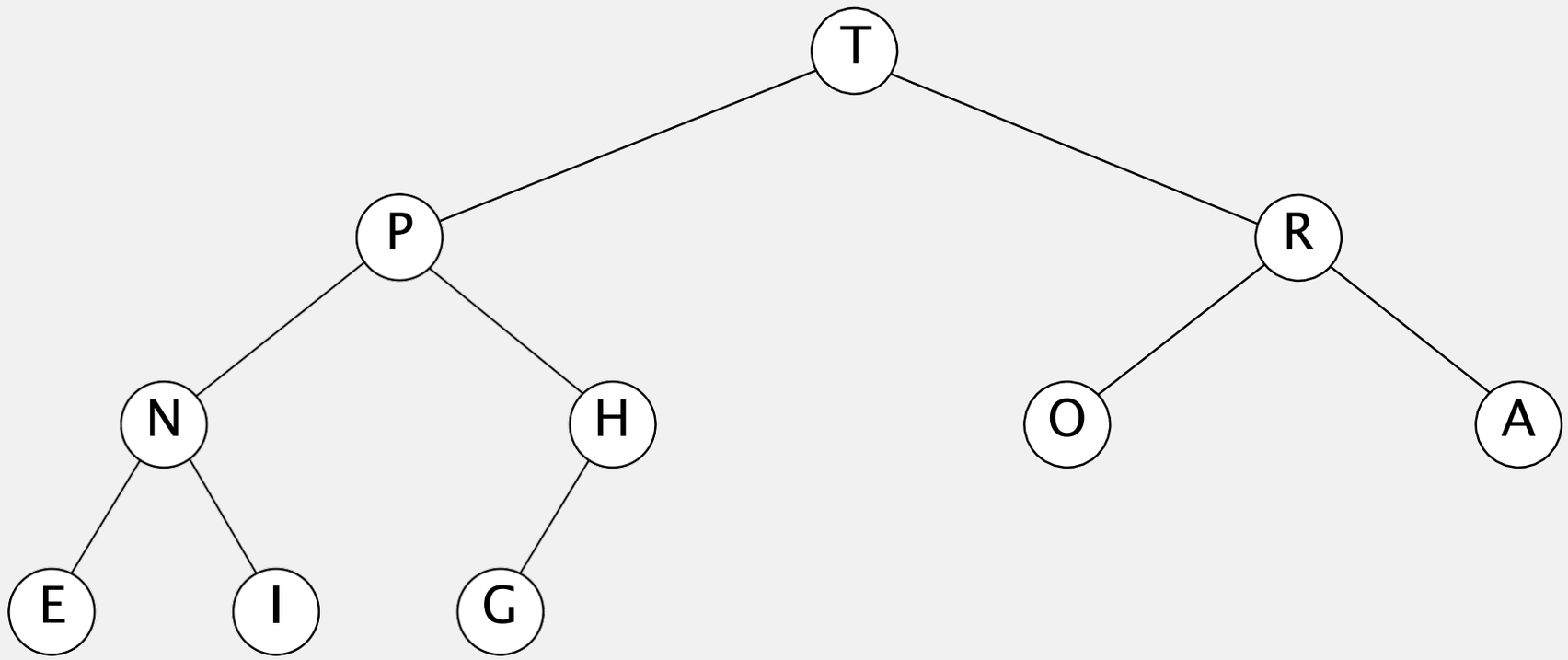
Heap paraqitja

Binary heap demo

Insert. Shtohet nyja në fund, pastaj “noton” lartë

Remove the maximum. Exchange root me nyjën në fund, pastaj “zhytet” poshtë.

heap e sortuar





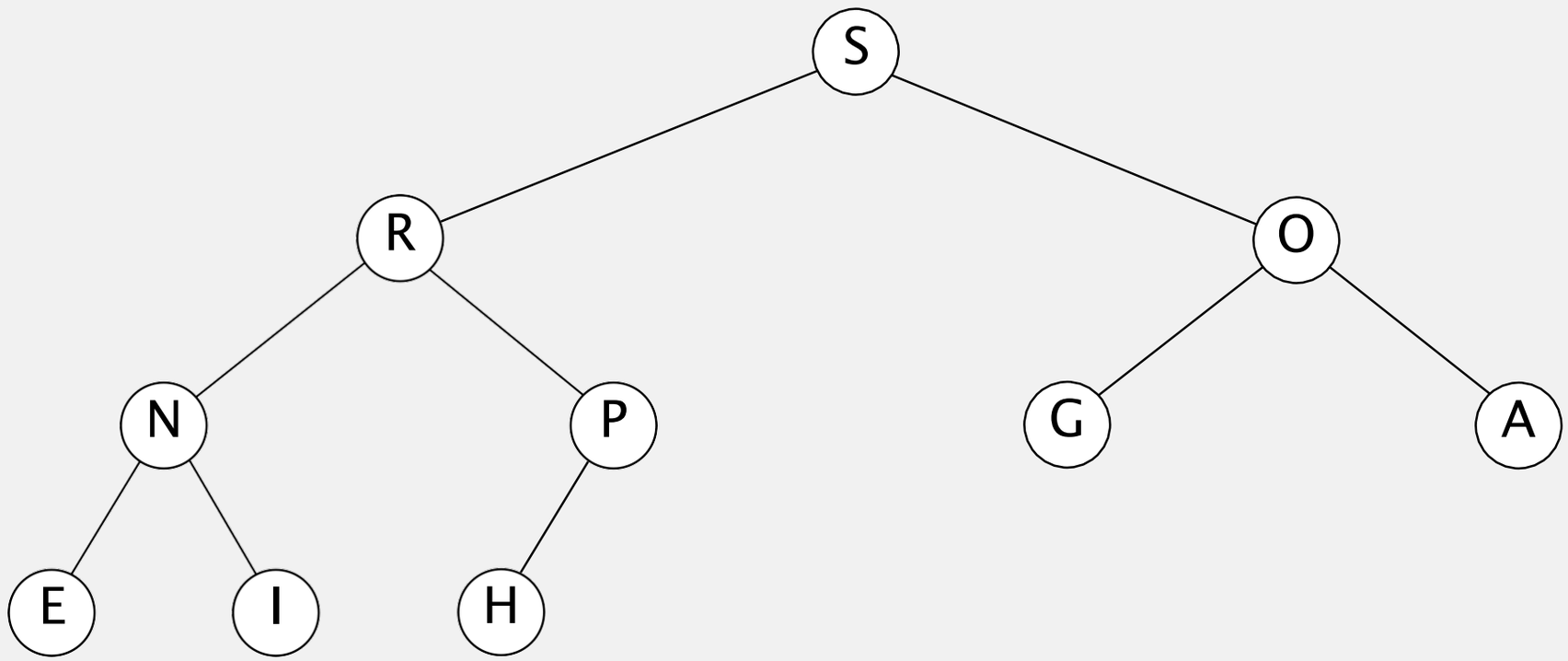
T	P	R	N	H	O	A	E	I	G	

Binary heap demo

Insert. Shtohet nyja në fund, pastaj “noton” lartë

Remove the maximum. Exchange root me nyjën në fund, pastaj “zhytet” poshtë.

heap ordered



S	R	O	N	P	G	A	E	I	H	

Përparimi në një heap

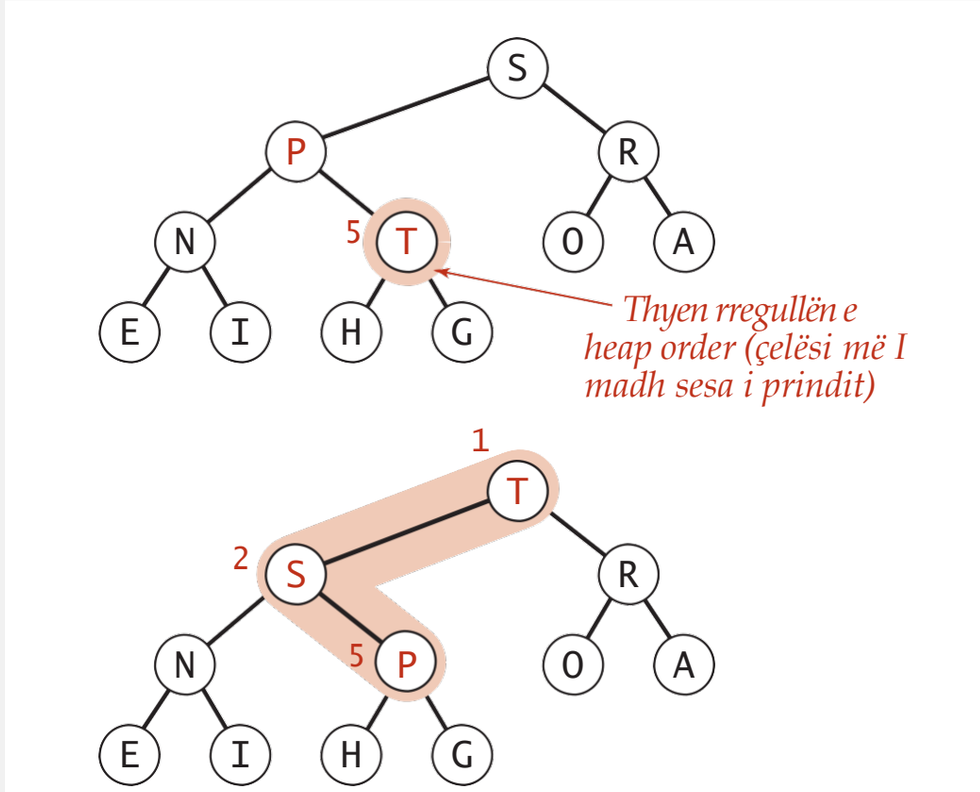
Skenari. Çelësi i fëmiut bëhet më i madh sesa çelësi i prindit të vet.

Për ta eliminuar thyerjen e rregullave:

- Exchange çelësin e fëmiut me çelësin e prindit.
- Përsëritet derisa të kthehet heap order.

```
private void swim(int k)
{
   while (k > 1 && less(k/2, k))
   {
      exch(k, k/2);
      k = k/2;
   }
}
```

parent of node at k is at k/2

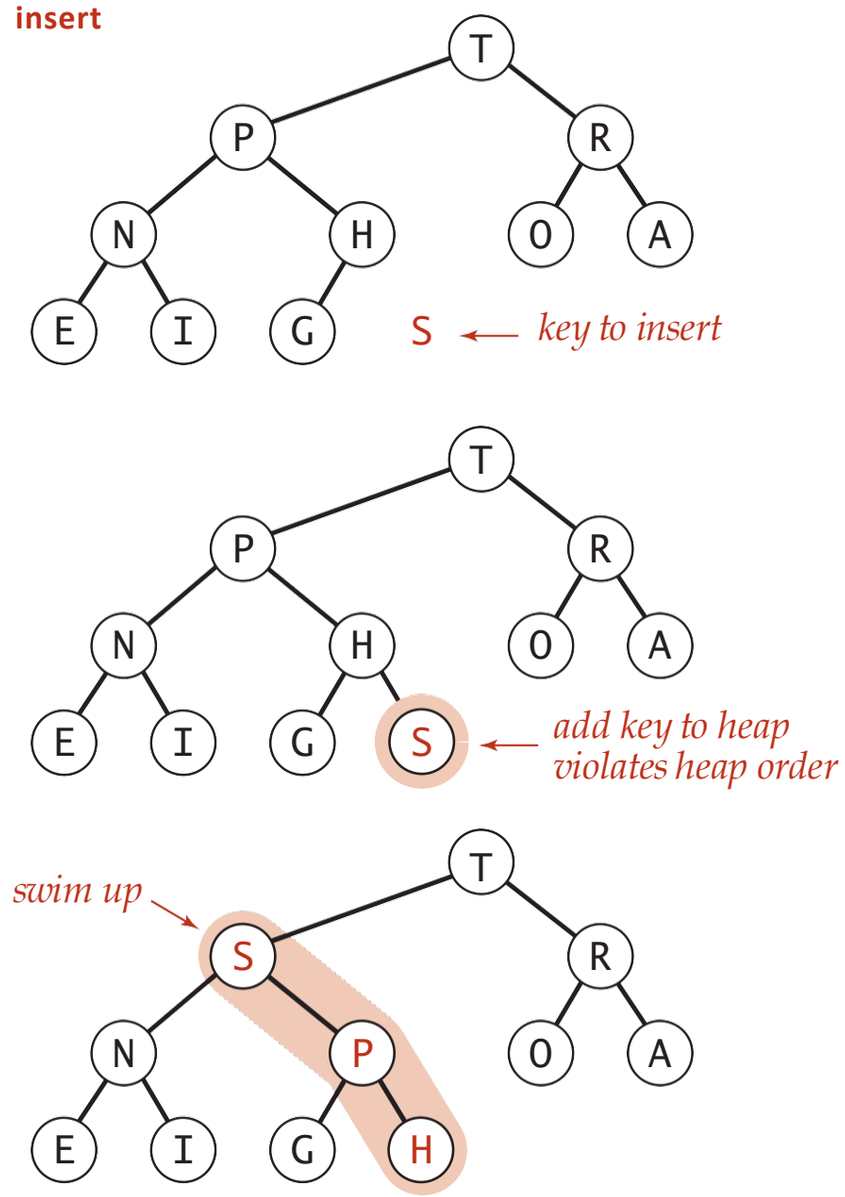


Insertimi në një heap

Insert. Add nyjën në fund, pastaj noton lartë.

Çmimi. Më së shumti ka $1 + \lg N$ krahasime.

```
public void insert(Key x)
{
   pq[++N] = x;
   swim(N);
}
```



Ulja në një heap

Skenari. Çelësi bëhet më i vogël sesa njëri (ose të dy) fëmijët e tij.

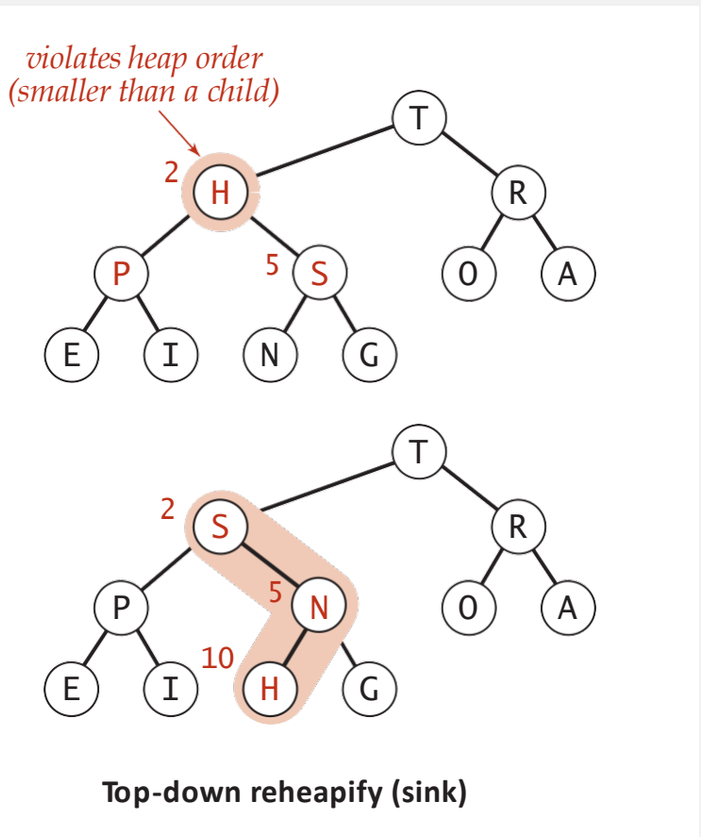
Për ta eliminuar thyerjen e rregullave :

why not smaller child?

- Exchange çelësin e prindit me çelësin e fëmiut më të madh.
- Përsëritet derisa të kthehet heap order.

```
private void sink(int k)
{
   while (2*k <= N)
   {
      int j = 2*k;
      if (j < N && less(j, j+1)) j++;
      if (!less(k, j)) break;
      exch(k, j);
      k = j;
   }
}
```

children of node at k are 2k and 2k+1



Top-down reheapify (sink)

Delete the maximum në një heap

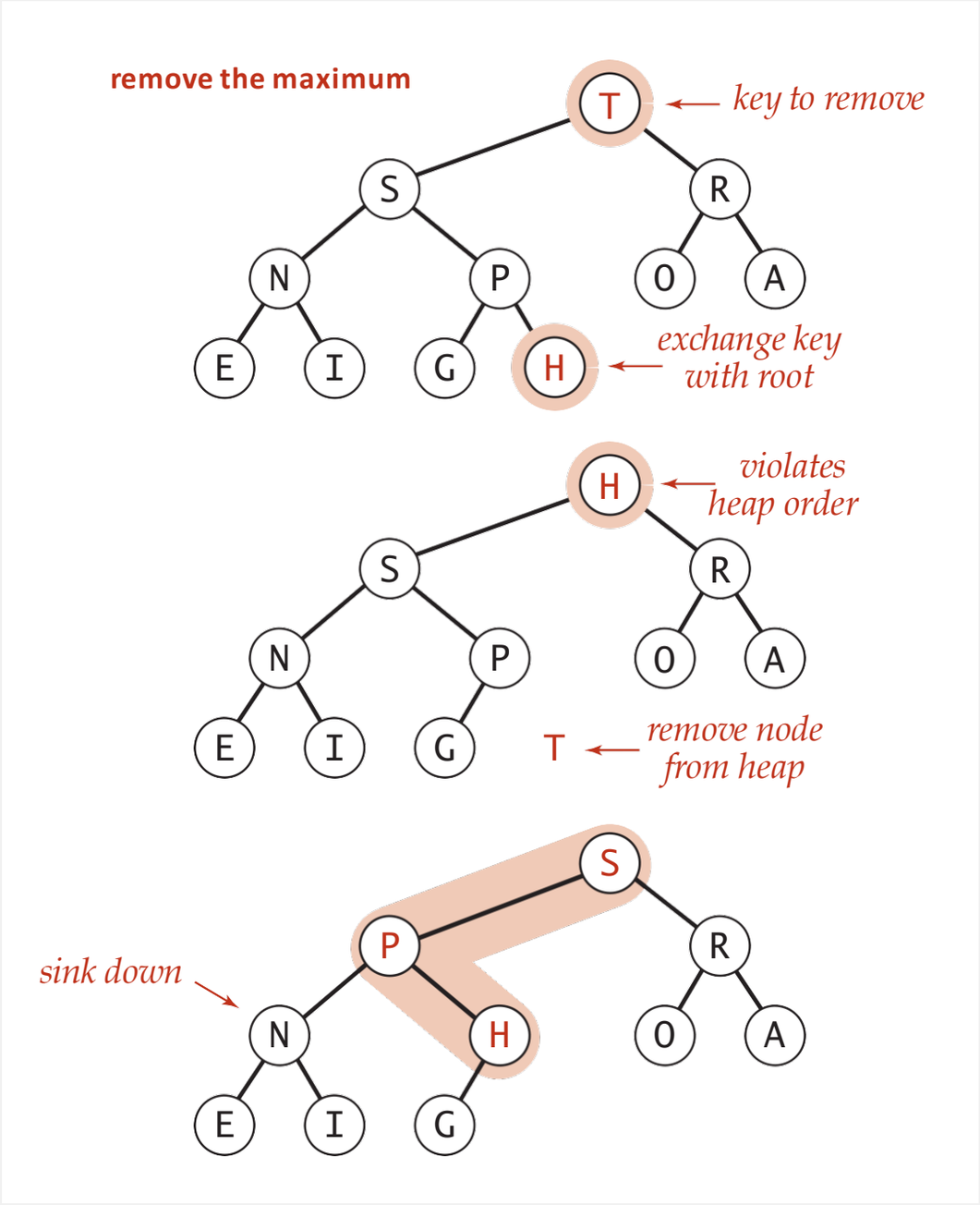
Delete max. Exchange root me nyjën në fund,pastaj zhytet poshtë

Çmimi. Më së shumti $2 \lg N$ krahasime.

```
public Key delMax()
{
   Key max = pq[1];
   exch(1, N--);
   sink(1);

}  pq[N+1] = null;
   return max;
```

prevent loitering



Binary heap: Java implementimi

```
public class MaxPQ<Key extends Comparable<Key>>
{
   private Key[] pq;
   private int N;

   public MaxPQ(int capacity)
   {  pq = (Key[]) new Comparable[capacity+1];  }

   public boolean isEmpty()
   {   return N == 0;   }
   public void insert(Key key)
   public Key delMax()
   {   /* see previous code */  }

   private void swim(int k)
   private void sink(int k)
   {   /* see previous code */  }

   private boolean less(int i, int j)
   {   return pq[i].compareTo(pq[j]) < 0;  }
   private void exch(int i, int j)
   {   Key t = pq[i]; pq[i] = pq[j]; pq[j] = t;  }
}
```

fixed capacity (for simplicity)

PQ ops

heap helper functions

array helper functions

Priority queues implementimi – përmbledhja e çmimit

implementation	insert	del max	max
unordered array	1	N	N
ordered array	N	1	1
binary heap	$\log N$	$\log N$	1

order- of- growth of running time for priority queue with N items

Priority queues implementimi – përmbledhja e çmimit

implementation	insert	del max	max
unordered array	1	N	N
ordered array	N	1	1
binary heap	$\log N$	$\log N$	1
d-ary heap	$\log_d N$	$d \log_d N$	1
Fibonacci	1	$\log N$ †	1
Brodal queue	1	$\log N$	1
impossible	1	1	1

← why impossible?

† amortized

order- of- growth of running time for priority queue with N items

Immutability: implementimi në Java

Tipi i dhënave. Bashkësi e vlerave dhe operacioneve në ato vlera.

Tipi i dhënave Immutable. Nuk mund të ndërrohet tipi i të dhënave pasi të krijohet.

```
public final class Vector {
   private final int N;
   private final double[] data;

   public Vector(double[] data) {
      this.N = data.length;
      this.data = new double[N];
      for (int i = 0; i < N; i++)
         this.data[i] = data[i];
   }

   …
}
```

can't override instance methods

instance variables private and final

defensive copy of mutable instance variables

instance methods don't change instance variables

Immutable. String, Integer, Double, Color, Vector, Transaction, Point2D.

Mutable. StringBuilder, Stack, Counter, Java array.

Immutability: vetitë

Tipi i dhënave Bashkësi e vlerave dhe operacioneve në ato vlera.

Tipi i dhënave Immutable. Nuk mund të ndërrohet tipi i të dhënave pasi të krijohet

Përparësitë.

- Thjeshtëson debugging.
- Më i sigurtë nga hostile code (kod i jashtëm dashakeq).
- Thjeshtëson concurrent programming.

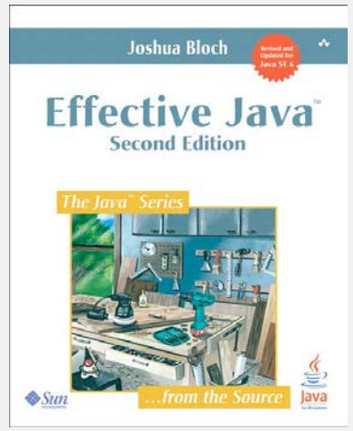


- Sigurtë të përdoret si çelës në priority queue ose symbol table.

Mangësitë. Duhet të (krijohet) create new object për çdo vlerë të tipit të dhënave.

“ Classes should be immutable unless there's a very good reason to make them mutable.… If a class cannot be made immutable, you should still limit its mutability as much as possible. ”

— *Joshua Bloch (Java architect)*





2.4 Priority Queues

- *API and elementary implementations*
- *binary heaps*
- *heapsort*
- *event-driven simulation*

Sortimi me një binary heap

```
public void sort(String[] a)
{
    int N = a.length;
    MaxPQ<String> pq = new MaxPQ<String>();
    for (int i = 0; i < N; i++)
        pq.insert(a[i]);
    for (int i = N-1; i >= 0; i--)
        a[i] = pq.delMax();
}
```

Q. Cilat janë vetitë?

A. $N \log N$, extra array me length N, nuk është stabil.

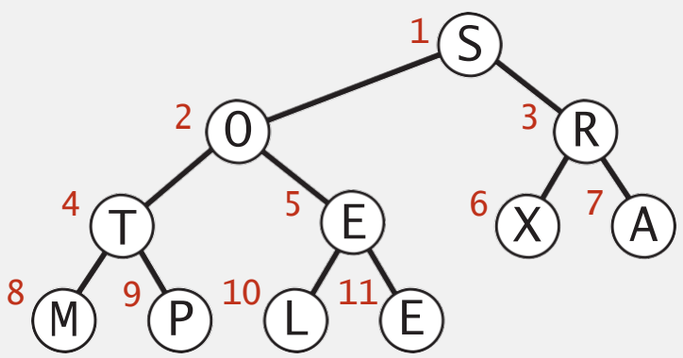
Heapsort intuita. A heap është një array.

Heapsort

In place sortimi me heap.

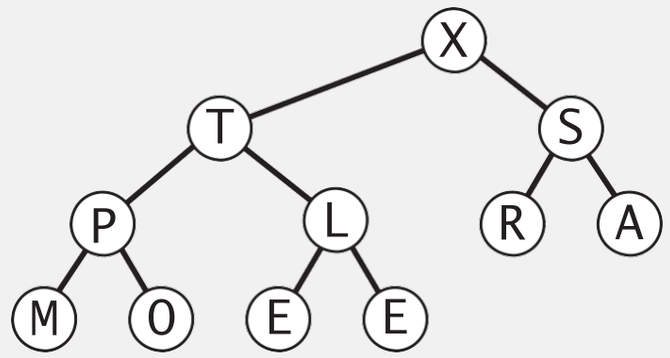
- Mirret input array si një complete binary tree.
- Heap konstruktimi: ndërtohet një max-heap me të gjithë N çelësat.
- Sortdown: vazhdimisht remove the maximum key.

keys in arbitrary order



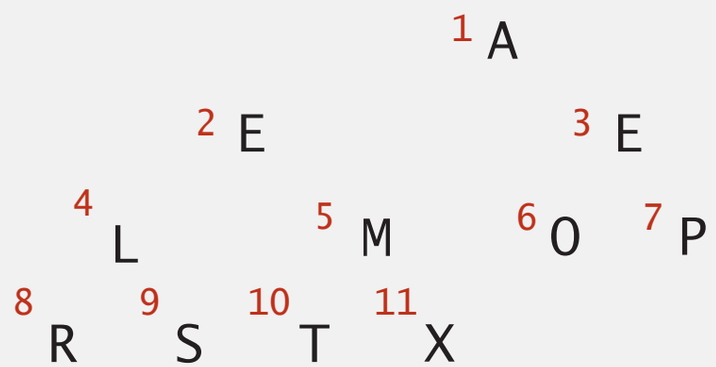
1	2	3	4	5	6	7	8	9	10	11
S	O	R	T	E	X	A	M	P	L	E

build max heap (in place)



1	2	3	4	5	6	7	8	9	10	11
X	T	S	P	L	R	A	M	O	E	E

sorted result (in place)



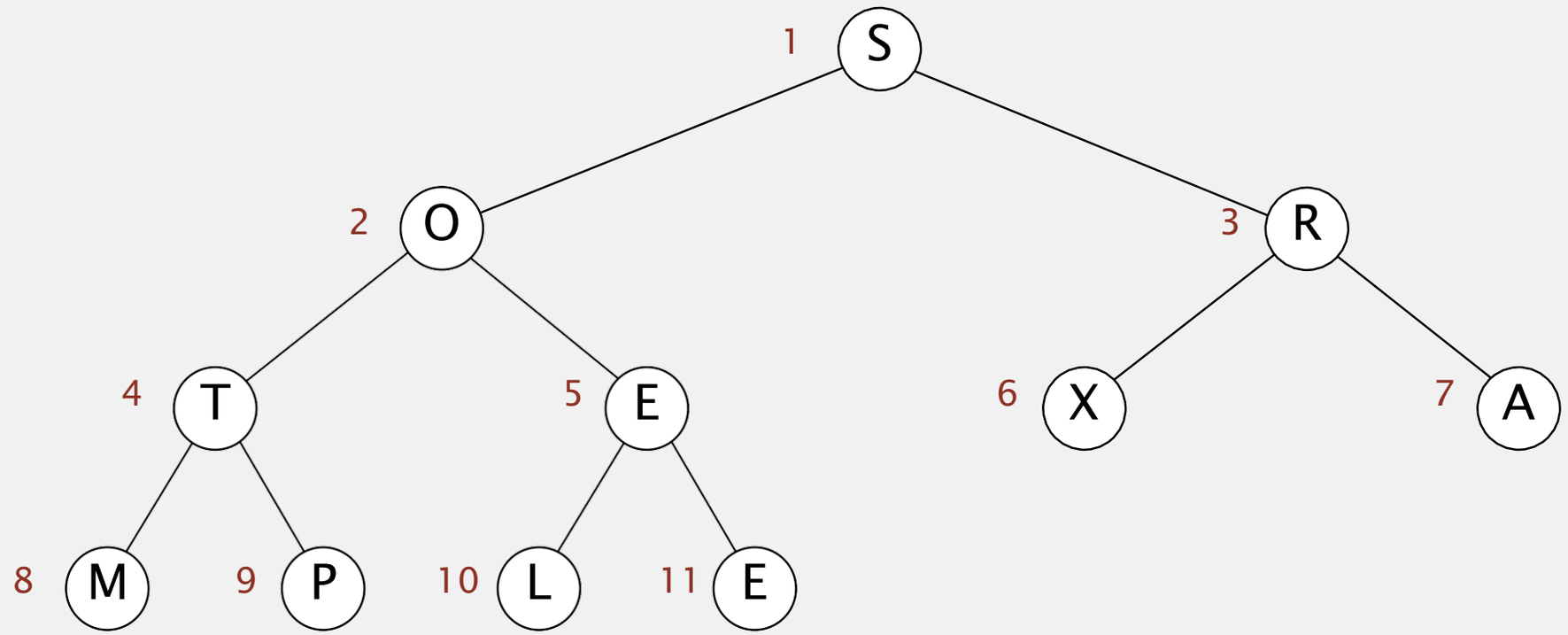
1	2	3	4	5	6	7	8	9	10	11
A	E	E	L	M	O	P	R	S	T	X

Heapsort demo

Heap construction. Ndërtohet max heap duke përdorur teknikën bottom-up.

we assume array entries are indexed 1 to N

array in arbitrary order





S	O	R	T	E	X	A	M	P	L	E
1	2	3	4	5	6	7	8	9	10	11

Heapsort demo

Sortdown. Vazhdimisht delete elementin me të madh të mbetur.

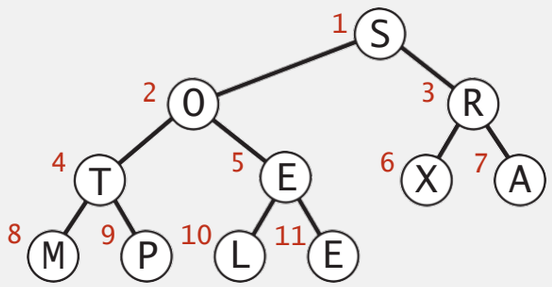
array in sorted order



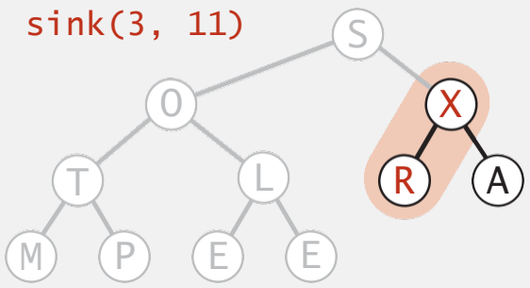
Heapsort: Konstruktimi i heap

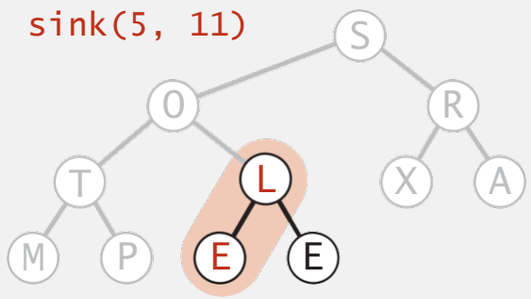
Kalimi i parë. Ndërtohet heap duke përdorur teknikën bottom-up.

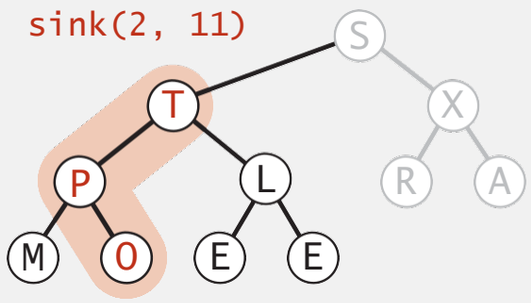
```
for (int k = N/2; k >= 1; k--)
   sink(a, k, N);
```

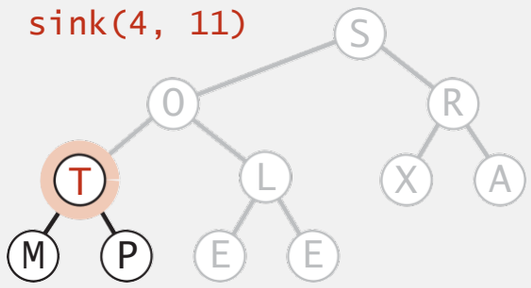


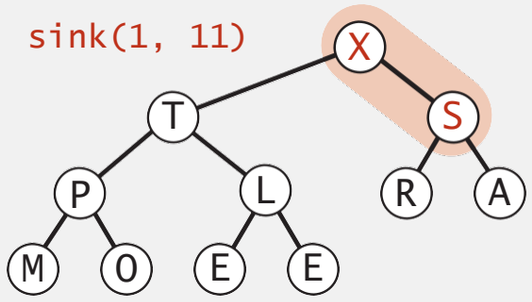
starting point (arbitrary order)











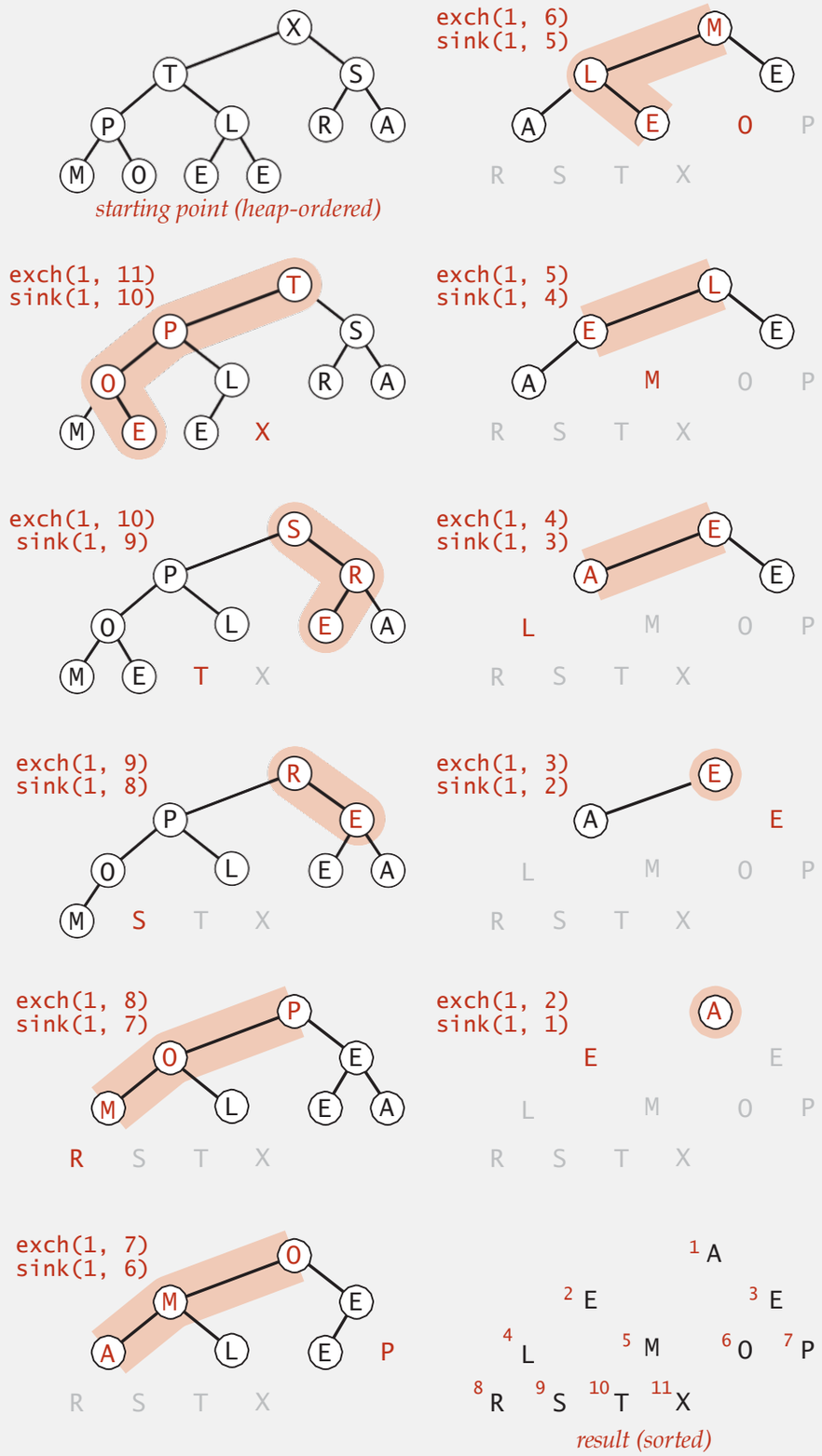
result (heap-ordered)

Heapsort: sortdown

Kalimi i dytë.

- Remove the maximum, një nga një.
- Vendoset në array, nuk bëhet null.

```
while (N > 1)
{          1, N--);
   exch(a,
   sink(a, 1, N);
}
```



Heapsort: Java implementimi

```
public class Heap
{
   public static void sort(Comparable[] a)
   {
      int N = a.length;
      for (int k = N/2; k >= 1; k--)
         sink(a, k, N);
      while (N > 1)
      {
         exch(a, 1, N);
         sink(a, 1, --N);
      }
   }

   private static void sink(Comparable[] a, int  k, int N)
   {  /* as before */  }

   private static boolean less(Comparable[] a, int i, int j)
   {  /* as before */  }

   private static void exch(Object[] a, int i, int j)
   {  /* as before */  }

}
```

but make static (and pass arguments)

but convert from 1-based indexing to 0-base indexing

Heapsort: trace

		a[i]											
N	k	0	1	2	3	4	5	6	7	8	9	10	11
initial values			S	O	R	T	E	X	A	M	P	L	E
11	5		S	O	R	T	L	X	A	M	P	E	E
11	4		S	O	R	T	L	X	A	M	P	E	E
11	3		S	O	X	T	L	R	A	M	P	E	E
11	2		S	T	X	P	L	R	A	M	O	E	E
11	1		X	T	S	P	L	R	A	M	O	E	E
heap-ordered			X	T	S	P	L	R	A	M	O	E	E
10	1		T	P	S	O	L	R	A	M	E	E	X
9	1		S	P	R	O	L	E	A	M	E	T	X
8	1		R	P	E	O	L	E	A	M	S	T	X
7	1		P	O	E	M	L	E	A	R	S	T	X
6	1		O	M	E	A	L	E	P	R	S	T	X
5	1		M	L	E	A	E	O	P	R	S	T	X
4	1		L	E	E	A	M	O	P	R	S	T	X
3	1		E	A	E	L	M	O	P	R	S	T	X
2	1		E	A	E	L	M	O	P	R	S	T	X
1	1		A	E	E	L	M	O	P	R	S	T	X
sorted result			A	E	E	L	M	O	P	R	S	T	X

Heapsort trace (array – përmbajtja pas çdo zhytje (sink)

Sorting algorithms: përmbledhje

	inplace?	stable?	best	average	worst	remarks
selection	✓		$\frac{1}{2} N^2$	$\frac{1}{2} N^2$	$\frac{1}{2} N^2$	N exchanges
insertion	✓	✓	N	$\frac{1}{4} N^2$	$\frac{1}{2} N^2$	use for small N or partially ordered
shell	✓		$N \log_3 N$	?	$c\, N^{3/2}$	tight code; subquadratic
merge		✓	$\frac{1}{2} N \lg N$	$N \lg N$	$N \lg N$	$N \log N$ guarantee; stable
timsort		✓	N	$N \lg N$	$N \lg N$	improves mergesort when preexisting order
quick	✓		$N \lg N$	$2 N \ln N$	$\frac{1}{2} N^2$	$N \log N$ probabilistic guarantee; fastest in practice
3-way quick	✓		N	$2 N \ln N$	$\frac{1}{2} N^2$	improves quicksort when duplicate keys
heap	✓		N	$2 N \lg N$	$2 N \lg N$	$N \log N$ guarantee; in-place
?	✓	✓	N	$N \lg N$	$N \lg N$	holy sorting grail